SPIKA TECH™

PROCEDIMIENTO DE DESARROLLO SEGURO

HISTORIAL DEL DOCUMENTO			
Versión	**Resumen de modificaciones**	**Fecha de entrada**	**Sustituye a (Código, revisión)**
01	Primera versión del documento.	12/03/2025	N/A

REGISTRO DE FIRMAS			
Función:	**Elaborado por:**	**Revisado por:**	**Aprobado por:**
Departamento:	Responsable del SGSI	Dirección	Jefe de seguridad (o suplente)
Nombre:	David Pozo	Carlos Rodrigo	Carlos Zúñiga
Firma:	Firmado por POZO SANCHEZ DAVID - ***6992** el día 12/03/2025	Firmado por RODRIGO RIVERO, CARLOS (AUTENTICACIÓN)	Firmado por ***8264** CARLOS ZUÑIGA (R: ****8690*) el día 13/03/2025
Fecha:	12/03/2025	12/03/2025	12/03/2025

CONTENIDO

1. OBJETIVO 5
2. ALCANCE 5
3. DEFINICIONES 5
4. INTRODUCCIÓN AL DESARROLLO SEGURO 5
5. CUMPLIMIENTO DE POLÍTICAS DE SPIKA TECH 7
6. REQUISITOS GENERALES 7
Información confidencial 7
Mínimo privilegio 8
Componentes de terceros 8
Protección de los datos de prueba 9
Metodología de desarrollo seguro 10
Aspectos de seguridad en el diseño integral del sistema 11
7. DESARROLLO SEGURO 11
Fases de análisis 12
Fase de investigación y diseño 15
Fase de implementación 18
Fase de pruebas 32
Fase de mantenimiento 33
8. ACEPTACIÓN Y PUESTA EN SERVICIO 33
Evidencias a entregar y aprobación del responsable 34
Pruebas de seguridad a realizar al componente software 34
Documentación del componente software 35
Entorno de pruebas o preproducción 36
Verificación de seguridad en la interrelación del componente software con el resto de las componentes del sistema 36
9. DESARROLLO EXTERNALIZADO 37
10. DOCUMENTACIÓN COMPLEMENTARIA 37
11. COMUNICACIÓN DE DEFICIENCIAS DEL PROCEDIMIENTO 38
12. REFERENCIAS 38

Anexo I. Análisis de riesgos 39

Anexo II. Checklist requisitos de seguridad 40

1. OBJETIVO

El objeto de este procedimiento es recoger de forma integral las acciones requeridas para el desarrollo seguro de aplicaciones en el alcance del sistema de información de SPIKA TECH.

Este documento se rige en definición, desarrollo y circuito de aprobación por la Norma de gestión de la documentación.

2. ALCANCE

Lo dispuesto en el presente documento será de aplicación a los sistemas de información responsabilidad de SPIKA TECH (en adelante, la Organización) en el ámbito de aplicación de las normas y estándares reconocidos en seguridad de la información.

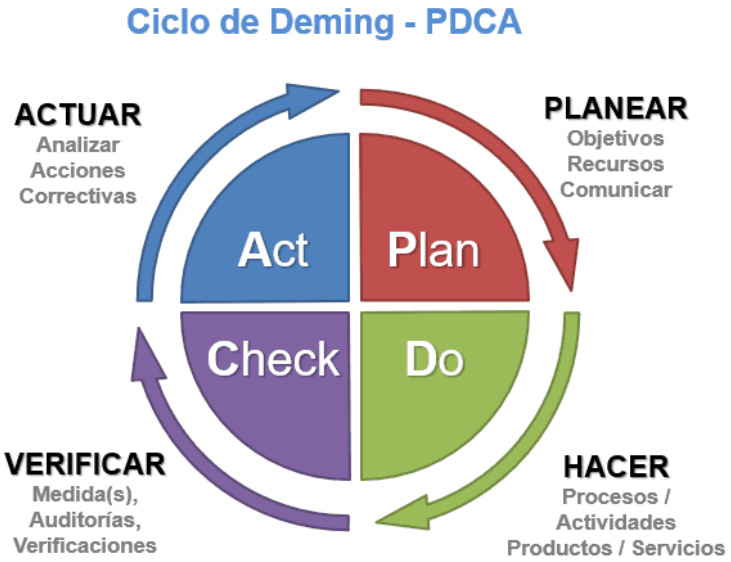
3. DEFINICIONES

ACRÓNIMO / TÉRMINO	DESCRIPCIÓN
Desarrollo seguro	Modelo de trabajo que se basa en la realización de chequeos de seguridad continuos del proyecto en construcción, incluso desde sus fases iniciales y antes de que se escriba una sola línea de código.
Framework	Conjunto estandarizado de conceptos, prácticas y criterios para enfocar un tipo de problemática particular que sirve como referencia, para enfrentar y resolver nuevos problemas de índole similar.

4. INTRODUCCIÓN AL DESARROLLO SEGURO

El control de seguridad relacionado con el cumplimiento del desarrollo seguro de software forma parte de la gestión integral de la seguridad de la información en la Organización. La seguridad es un proceso continuo, no un producto estático.

Al igual que otros procesos internos, como el propio desarrollo de software, es recomendable abordar el control del desarrollo seguro mediante una planificación estructurada en iteraciones o fases diferenciadas. Esto permite priorizar de manera gradual las acciones más críticas por realizar y definir, de forma precisa, qué aspectos pueden abordarse en cada iteración. A continuación, se muestra un gráfico sencillo que ilustra este concepto utilizando el Ciclo de Deming.



El contenido del presente documento no exime en ningún caso de otras responsabilidades inherentes a la seguridad de la Organización, como por ejemplo disponer de políticas a instrucciones específicas para el cumplimiento de los acuerdos con proveedores estratégicos de desarrollo.

El presente documento, resume, introduce y complementa, las directrices marcadas por las siguientes organizaciones y estándares de marco nacional o internacional referentes al desarrollo seguro de aplicaciones:

- Esquema Nacional de Seguridad - https://ens.ccn.cni.es/es/
- ISO 27001:2022.
- Centro Criptológico Nacional - https://www.ccn-cert.cni.es/
- Instituto Nacional de Ciberseguridad - https://www.incibe.es/
- Proyecto OWASP (Open Web Application Security Project) - https://owasp.org/about/

Con el objetivo de facilitar el cumplimiento de los requisitos de seguridad durante todo el ciclo de vida del desarrollo seguro de aplicaciones el presente documento se dividirá en dos secciones principales:

- La primera abordará las medidas generales necesarias para cumplir con los requisitos de diferentes estándares de seguridad (ISO 27001, ENS, OWASP,etc) en lo referente al desarrollo seguro de aplicaciones, proporcionando un marco común para garantizar la protección adecuada de los sistemas y la información.
- La segunda sección estará dedicada a las medidas específicas que deben implementarse en cada una de las fases del ciclo de desarrollo seguro de aplicaciones, detallando las acciones concretas a seguir en cada etapa para asegurar la integridad y seguridad del software.

5. CUMPLIMIENTO DE POLÍTICAS DE SPIKA TECH

En caso de que el desarrollo de aplicaciones sea externalizado, el proveedor contratado deberá adherirse estrictamente a las políticas de seguridad corporativas establecidas. Esto incluye, de manera particular, el cumplimiento de todas las normas y directrices relacionadas con el desarrollo seguro de software, garantizando que los estándares de seguridad sean respetados en todo el proceso de desarrollo y que se asegure la protección de la información y los sistemas involucrados.

Asimismo, cualquier tercero que realizase actividades de desarrollo de aplicaciones para SPIKA TECH deberá, a requerimiento de este, aportar evidencias del cumplimiento de los diferentes requisitos de seguridad. Estas evidencias se encuentran referenciadas en el presente documento.

6. REQUISITOS GENERALES

Se describen, a continuación, los puntos principales a revisar y cumplir, sobre los procesos de desarrollo de software llevados a cabo por la organización encargada del mismo, para la adecuación y cumplimiento de la ISO 27001.

Información confidencial

Para cada proyecto de desarrollo de software, de forma independiente, deberán usarse diferentes entornos de desarrollo: Desarrollo (DEVELOPMENT), Preproducción (PRE-PRODUCTION) y Producción (PRODUCTION).

Los entornos de desarrollo y producción deberán estar diferenciados, debiéndose prestar especial atención a los siguientes aspectos:

- Se recomienda que, en las medidas de las posibilidades, las contraseñas y claves empleadas en el despliegue a producción, sean completamente nuevas, generadas de forma segura y desde cero, no teniendo ninguna relación con las contraseñas y claves usadas en entornos anteriores. En todo caso, los usuarios con privilegios de despliegue a producción deberán tener activos los registros de auditoría y serán monitorizados periódicamente.
- Los entornos de desarrollo y preproducción deben tener una configuración lo más similar posible a la de los entornos de producción, con objeto de que los resultados de las pruebas de validación llevadas a cabo en dichos entornos se ajusten a las obtenidas en los entornos operativos.
- Ningún código o dato con motivación de pruebas debe pasarse al entorno de preproducción y producción, siendo todos estos eliminados en la subida a la rama de preproducción y producción, quedándose exclusivamente en entornos anteriores, como el de desarrollo.
- Los controles de seguridad implantados en los entornos de desarrollo y preproducción deben ser equivalentes y proporcionales a los implantados en los entornos de producción, en función de la información tratada en dichos entornos, especialmente si se emplean datos de prueba reales.
- En los entornos de producción se prohibirá la existencia de herramientas y de todo software que no sea necesario para la ejecución de las aplicaciones productivas. Tampoco estará en

el entorno de producción el código fuente de las aplicaciones, ubicándose en él únicamente el código ejecutable.

Evidencias:

- **Registros de Generación de Claves**: evidencia de la generación de contraseñas y claves únicas para el entorno de producción, como registros de la herramienta o proceso utilizado para la creación de estas claves (por ejemplo, logs de software de gestión de claves, parámetros o políticas de cambios de clave, scripts de generación de claves seguras).
- **Prácticas de Rotación de Claves**: documentación que muestre cómo se implementan prácticas de rotación y gestión segura de claves en cada entorno, con fechas de cambios y responsables.
- **Captura de pantalla de los diferentes entornos utilizados**.

Mínimo privilegio

En seguridad de la información, ciencias de la computación y otros campos, el principio de mínimo privilegio (también conocido como el principio de menor autoridad) indica que, en una particular capa de abstracción de un entorno computacional, cada parte (como puede ser un proceso, un usuario o un programa, dependiendo del contexto) debe ser capaz de acceder solo a la información y recursos que son necesarios para su legítimo propósito.

Lo anterior significa que cada rol debe poder realizar lo estrictamente necesario y no más. Deben revisarse los componentes software que se están desarrollando y los propios procesos empleados para desarrollarlos, desde este principio de seguridad.

Evidencias:

- **Evidencias de gestión de usuarios.**
- **Evidencias de grupos de usuarios y roles** que implementa una aplicación, ya sea en su documento de diseño, como en la interfaz gráfica del producto final.

Estas evidencias deben complementarse con las referentes a mínimo privilegio y segregación de funciones del apartado 7.2.1. Principios del diseño seguro de software.

Componentes de terceros

Se considera software de terceros a cualquier aplicación, librería, componente, código fuente, etc. que no es propiedad de la Organización y es utilizado en cualquier fase del proyecto, forme parte o no del producto final entregado.

Deben cumplirse las siguientes medidas:

- Únicamente debe utilizarse software de terceros que se encuentre autorizado, correctamente licenciado y que cumpla las políticas de seguridad corporativas.

- En el caso de software o componentes utilizados por la aplicación desarrollada, se debe configurar con un nivel de seguridad lo más restrictivo posible que permita su uso con las finalidades previstas.
- Se debe utilizar la versión del software más actualizada posible o, al menos, la que posea los últimos parches de seguridad instalados.

Evidencias:

- **Listado de software autorizado**.
- **Capturas de Pantalla de la Versión Actual:** imágenes de la interfaz de administración del software mostrando la versión actual instalada confirmando que es la más reciente.

Protección de los datos de prueba

Debido a los riesgos asociados al uso de datos reales en entornos de prueba, es fundamental seguir las normativas establecidas por el RGPD, en particular el artículo 32 del RGPD referente a la seguridad del tratamiento, que obliga al responsable y al encargado del tratamiento a implementar medidas técnicas y organizativas apropiadas que garanticen un nivel de seguridad adecuado al riesgo para los derechos y libertades de los interesados.

En esta misma línea, el Supervisor Europeo de Protección de Datos (EDPS) establece en sus directrices que, durante la fase de prueba, debe evitarse el uso de datos personales reales, ya que esto podría exponer dichos datos a personas no autorizadas.

Es por ello, que se evitará el uso de datos reales en entornos de desarrollo y prueba. En la medida de la posible se deberán implantar procedimientos de disociación o enmascaramiento que imposibiliten la obtención de los datos originales. En caso de que el uso de datos reales sea necesario, se deben cumplir las siguientes medidas de seguridad:

- La copia de datos reales a los entornos de prueba debe ser autorizada por escrito por el propietario o responsable de dichos datos, como puede ser el Responsable de la Información y, en caso de tratarse de datos de naturaleza personal, por el Delegado de Protección de Datos. Para ello, se seguirá el **Procedimiento de gestión de autorizaciones**.
- En el caso de no ser posible el cumplimiento de las medidas de seguridad del entorno de producción en el entorno de preproducción o pruebas, para poder autorizar una extracción los datos éstos deberán ser previamente anonimizados / disociados.
- Cuando los datos a extraer del entorno de Producción a Preproducción o Pruebas sean de carácter personal de los clasificados como categorías especiales de datos según el RGPD, obligatoriamente deberán ser anonimizados / disociados con antelación, con independencia de las medidas de seguridad aplicadas.
- En cualquier caso, deberá utilizarse el mínimo conjunto de datos reales que sea necesario para poder llevar a cabo las pruebas.
- Los datos que se encuentren en los entornos de prueba se protegerán y controlarán de acuerdo a las medidas de seguridad implantadas en los entornos de producción y, en

particular, según las disposiciones de la legislación vigente en materia de protección de datos de carácter personal.

- Se deben borrar los datos reales de los entornos de prueba una vez haya concluido la finalidad para la que fueron copiados.

Evidencias:

- **Formularios de Solicitud de Autorización**: ejemplos de formularios de autorización por escrito firmados por el Responsable de la Información y el Delegado de Protección de Datos para la copia de datos reales.
- **Registros de Acceso al Entorno de Pruebas**: logs que demuestren quién ha accedido a los entornos de prueba y cuándo, asegurando que solo el personal autorizado tiene acceso.
- **Ejemplos de Datos Anonimizados**: muestras de datos antes y después del proceso de anonimización, mostrando cómo se ha protegido la información personal.

Metodología de desarrollo seguro

En este documento se propone el empleo de la metodología OWASP (Open Web Application Security Project), que deberá adecuarse a las posibilidades y requisitos de cada proyecto y a la organización que realice el desarrollo de aplicaciones, mediante el uso de los siguientes elementos:

- **OWASP Top Ten**

  Los OWASP Top 10, son documentos de concienciación estándar para desarrolladores y seguridad de aplicaciones. Resumen los 10 riesgos más frecuentes a tratar, se recomienda el descubrimiento de vulnerabilidades y la revisión de realización de contramedidas a estas, comenzando por los 10 riesgos expuesto por este listado. Referencias:

  - https://owasp.org/Top10/
  - https://owasp.org/www-project-mobile-top-10/
  - https://owasp.org/www-project-api-security/

- **OWASP SAMM (Software Assurance Maturity Model)**

  Aproximación a nivel estratégico para el desarrollo seguro de software en una organización. Con este modelo se puede cuantificar el nivel de madurez que lleva a cabo una organización en su implementación de desarrollo seguro.

  - Referencia: https://owasp.org/www-project-samm/

- **OWASP Code Review Guide**

  Las auditorías de código trabajan el descubrimiento de vulnerabilidades presentes en la organización, mediante la revisión del código. La Guía de revisión de código OWASP es un libro técnico escrito para los responsables de las revisiones de código (responsables de desarrollo, desarrolladores, profesionales de la seguridad).

- Referencia: https://owasp.org/www-project-code-review-guide/

Asimismo, también se tendrán en cuenta los principios y guías establecidos por el Esquema Nacional de Seguridad, referenciados en el Anexo II. Checklist requisitos de seguridad.

Aspectos de seguridad en el diseño integral del sistema

Para verificar que se tiene en cuenta la seguridad, desde el propio diseño de los componentes software, se propone revisar los puntos técnicos recogidos en el proyecto OWASP Application Security Verification Standard (ASVS), sobre los documentos de diseño de cada proyecto de desarrollo y posteriormente también sobre su implementación. A continuación, su detalle y referencia:

- **Checklist "OWASP Application Security Verification Standard (ASVS)"**

  El Proyecto del Estándar de Verificación de Seguridad de Aplicaciones de OWASP, que es un Checklist, proporciona una base para probar los controles técnicos de seguridad de las aplicaciones web y también proporciona a los desarrolladores una lista de requisitos para un desarrollo seguro. Referencias:

  - https://owasp.org/www-project-application-security-verification-standard/
  - https://mas.owasp.org/MASVS/

- Checklist sobre las medidas de seguridad exigidas por el Esquema Nacional de Seguridad. Referenciadas en el Anexo II. Checklist requisitos de seguridad.

7. DESARROLLO SEGURO

El objetivo del presente apartado es ofrecer una documentación técnica de consulta a seguir para cualquier actor implicado en el desarrollo de software para SPIKA TECH. Este documento sirve de guía para adoptar buenas prácticas y cumplir con los requerimientos de seguridad durante todo el ciclo de vida del desarrollo de software.

A continuación, se reflejan algunos posibles actores implicados en un proyecto de desarrollo de software, todos ellos con implicación en la seguridad del proyecto:

- Desarrollo de software: responsable de desarrollo, arquitecto, analistas (tanto funcionales como orgánicos), desarrolladores, administradores de bases de datos, etc.
- Control de calidad: responsable de control de calidad, analista QA, tester QA, usuarios de la aplicación, etc.
- Control de seguridad: responsable de seguridad gestionada, consultor de ciberseguridad, consultor de desarrollo seguro, auditor de seguridad, etc.
- Dirección del proyecto: dirección, product owner, responsable de gestión de proyecto, jefe de proyecto, cliente, etc.
- Operaciones y mantenimiento: responsable de sistemas y redes, administradores de sistema, operadores de infraestructura, etc.

En los siguientes capítulos se detallan las distintas fases que componen el ciclo de desarrollo seguro de software, abarcando cada etapa crítica para garantizar la seguridad y calidad del producto:

- Análisis: se llevará a cabo un exhaustivo análisis de riesgos para identificar vulnerabilidades potenciales.
- Investigación y Diseño: se establecerán las bases estructurales del sistema con un enfoque en la seguridad.
- Implementación: se desarrollará el código siguiendo las mejores prácticas de seguridad.
- Pruebas: se validará el funcionamiento del software mediante pruebas exhaustivas para identificar y corregir fallos.
- Mantenimiento: se garantizará que el sistema se mantenga seguro y actualizado frente a nuevas amenazas y vulnerabilidades.

Fases de análisis

En esta primera fase del proyecto, que corresponde al análisis, se lleva a cabo la recopilación de los requisitos que deberá cumplir el futuro software. Los analistas funcionales y orgánicos, mediante entrevistas con el project manager, apoyado en los desarrolladores, identifican y especifican estos requisitos (toma de requisitos). En este punto, un consultor de seguridad participa para analizar y validar que estos requisitos sean seguros, además de añadir aquellos que son específicamente de seguridad.

Durante la fase de análisis, el equipo de desarrollo de software (a veces con la participación temprana del equipo de control de calidad u otros colaboradores) organiza y revisa los requisitos analizados que deberá cumplir el futuro software.

Dado que estamos en la primera fase del ciclo de vida del desarrollo de software, estos requisitos, al ser previos a cualquier tipo de implementación o diseño, deben ser revisados desde el punto de vista de la seguridad una vez que estén completamente desarrollados o en una etapa avanzada de madurez. Normalmente, los requisitos del proyecto son revisados en términos de seguridad mientras se incorporan los requisitos específicos de seguridad. Es especialmente recomendable que esta revisión sea realizada por personal especializado.

7.1.1 Análisis de riesgos

Previo al desarrollo de nuevas aplicaciones, o modificaciones sustanciales en las existentes, éstas deben ser analizadas para determinar los requisitos de seguridad que deben cumplir, así como establecerse plazos y planes para hacerlo.

En el desarrollo de una nueva aplicación, o en los cambios o evolutivos de una aplicación ya existente, debe realizarse un análisis de riesgos formal donde se cumplimente el modelo adjunto Anexo I. Análisis de Riesgos de modo que cada nueva aplicación o cambio/evolutivo debe tener documentado este análisis de riesgos donde se especifican los requisitos y medidas de seguridad que debe cumplir, especialmente para mitigar los riesgos evaluados como inaceptables.

Asimismo, este análisis de riesgos tendrá en cuenta las regulaciones en materia de protección de datos, en especial las directrices del Reglamento General de Protección de Datos (RGPD) de la Unión Europea, así como cualquier legislación nacional aplicable. Se prestará especial atención a

aquellas aplicaciones que traten datos englobados dentro de las categorías especiales de datos personales recogidas en el artículo 9 del RGPD.

El análisis de riesgos será realizado por el equipo de desarrollo encargado, quienes deberán usar la plantilla específica Anexo I. Análisis de Riesgos para llevar a cabo dicho análisis. Posteriormente, el Responsable de Seguridad de la Información de la Organización o el responsable de la aplicación designado por la Organización se encargará de revisar y validar el análisis de riesgos realizado, asegurándose de que cumple con los requisitos de seguridad establecidos y es adecuado para mitigar posibles vulnerabilidades.

Antes del despliegue en producción de un nuevo aplicativo desarrollado o de cambios/evolutivos que se realicen sobre los ya existentes, se verificara el grado de cumplimiento de los requisitos y medidas de seguridad que se definieron en el análisis de riesgos. Esta verificación debe quedar documentada, indicando los requisitos que no se están cumpliendo y estableciendo prioridades para su posterior resolución, de forma que se trace un plan de acción de cumplimiento de requisitos de seguridad. Dicho plan de acción debe tener en cuenta que la no implantación de los requisitos considerados de riesgo alto deberá ser corregida antes de la entrada a producción.

Con el objetivo de garantizar que la aplicación cumpla con las normativas de protección de datos y los estándares de seguridad aceptados, se incluirá el Anexo II. Checklist de requisitos de seguridad. Esta checklist servirá como una guía de verificación que abarca las principales áreas de seguridad, permitiendo al equipo de desarrollo externo y al Responsable de Seguridad validar que todos los controles necesarios han sido implementados de acuerdo con la normativa vigente y las mejores prácticas.

7.1.2 Referencia general a requisitos de seguridad del proyecto OWASP Application Security Verification Standard (ASVS)

Aunque, como marco general, se utilizará la metodología y los requisitos establecidos en el Anexo II para realizar un desarrollo seguro, en este punto destacamos el estándar internacional del proyecto OWASP, el cual ya dispone de una lista de requerimientos genérica (ASVS) para tener en cuenta como punto de partida y referencia.

Los estándares de verificación de seguridad de aplicaciones del proyecto OWASP Application Security Verification Standard (ASVS), consisten en una lista de requisitos o pruebas de seguridad de aplicaciones que los arquitectos, desarrolladores, evaluadores, profesionales de seguridad, proveedores de herramientas y usuarios pueden usar para definir, construir, probar y verificar aplicaciones de seguridad.

La implementación de ASVS tiene dos objetivos principales:

- Ayudar a las organizaciones a desarrollar y mantener aplicaciones seguras.
- Permitir a los proveedores de servicios de seguridad, proveedores de herramientas de seguridad y consumidores a establecer los requisitos necesarios.

ASVS establece tres niveles de cumplimiento, estos varían dependiendo de la profundidad en la seguridad aplicada:

- El nivel 1 de ASVS se utiliza para componentes software que ofrecen servicio en contextos que no requieren de seguridad, siendo considerados un nivel de seguridad bajo. Este nivel

no ofrece mucha protección y las pruebas de seguridad como un test de penetración suelen ofrecer resultados significativos.

- El nivel 2 de ASVS es adecuado para cualquier aplicación que realiza un tratamiento de datos sensibles. Refiriéndonos a datos sensibles como datos que no sean estrictamente públicos y tengan algún nivel de clasificación de la información, como por ejemplo información interna, confidencial o secreta. Éste es el nivel recomendado y que aplica al contexto de la mayoría de aplicaciones.
- El nivel 3 de ASVS es el indicado si los componentes software están ofreciendo servicio en un contexto crítico, este contexto es el que engloba a sistemas informáticos que realizan transacciones de alto valor para una organización, aplicaciones que realizan tratamiento de datos muy sensibles como por ejemplo los datos médicos, o sistemas que requieren ofrecer la mayor garantía de confianza.

Cada nivel ASVS contiene una lista general con los requisitos de seguridad que un proyecto de desarrollo de software debe de cumplir para ese contexto. Cada uno de estos requisitos de seguridad se organiza específicamente dentro de un área del proyecto o funcionalidad del futuro componente software, por lo que es fácilmente asignable a los diferentes actores que participan en el proyecto y a las diferentes capacidades que ofrece el producto software.

El nivel 1 de ASVS es el único que no requiere de procesos internos y puede ser probado y confirmado su cumplimiento a través de personas sin acceso ni conocimientos específicos de la organización, desde el exterior. Este nivel 1 sólo dispone pruebas llevadas a cabo de forma externa de tipo caja negra, lleva a cabo el control de seguridad por parte de la organización en una forma reactiva (en lugar de proactiva), por lo que no es un proceso que ofrezca una protección del sistema eficaz, esta es la causa por la que está desaconsejado para cualquier contexto que requiera una seguridad adecuada. Los niveles posteriores 2 y 3 disponen de un control de seguridad proactivo y requieren el acceso a la documentación, código fuente, configuración del sistema y entrevistas con el personal involucrado en el proceso de desarrollo de software.

Los requisitos de seguridad que ofrece ASVS serán usados como referencia para la ejecución de las siguientes tareas:

- Guía de cumplimiento de seguridad en la arquitectura del componente software.
- Checklist de código seguro, para las revisiones de seguridad sobre el código estático.
- Guía de pruebas de seguridad dinámicas, unitarias y de integración.
- Guía de entrenamiento en desarrollo seguro.
- Guía de control de seguridad en desarrollo con metodologías ágiles.
- Guía de marco de trabajo para la producción de productos software seguros.

A continuación, se exponen las categorías que dividen todos los controles de verificación de requisitos de seguridad. Dependiendo del componente software que desarrolle el proyecto algunos controles aplicarán y otros no:

- Arquitectura diseño y modelo de amenazas.
- Autenticación.

- Gestión de sesión.
- Control de acceso.
- Validación, subsanación y codificación de datos.
- Criptografía y almacenamiento.
- Gestión de errores y registro.
- Protección de datos.
- Comunicaciones.
- Código malicioso.
- Lógica de negocio.
- Archivos y recursos.
- API y servicios web.
- Configuración.

Todos los controles de verificación de requisitos descritos se encuentran documentados en el "OWASP Application Security Verification Standard", listos para su verificación en cualquier proyecto de desarrollo de software.

Evidencias:

- Análisis de Riesgos documentado.
- Metodología de desarrollo seguro seguido por la organización.

Fase de investigación y diseño

La seguridad es un componente esencial que debe integrarse desde el inicio del desarrollo de software. Implementar medidas de seguridad después de que el desarrollo ha concluido resulta costoso y requiere un esfuerzo significativo en términos de reingeniería. Por ello, la fase de investigación y diseño se convierte en un momento crucial para establecer un marco seguro que permita identificar y corregir posibles vulnerabilidades antes de que puedan ser explotadas.

Durante esta fase, se realiza una revisión exhaustiva del documento de arquitectura desde una perspectiva de ciberseguridad. Este análisis no solo busca identificar debilidades, sino también integrar de manera proactiva principios de seguridad en la estructura del diseño, garantizando que las decisiones arquitectónicas tomen en cuenta los riesgos desde el principio. La colaboración con equipos de seguridad o con expertos desde la fase de concepto es ideal para asegurar que la seguridad esté presente en cada etapa del desarrollo.

El proceso de revisión de arquitectura y diseño no se limita a la documentación, sino que también incluye la capacidad de los equipos de desarrollo e ingeniería para descomponer la aplicación e identificar elementos clave como límites de confianza, flujos de datos, puntos de entrada y código privilegiado. Comprender la configuración de despliegue físico de la aplicación y prestar especial atención a las áreas comúnmente vulnerables es esencial para construir una base segura.

La revisión de seguridad del diseño es conducida principalmente por un consultor de seguridad o un responsable de seguridad que no haya participado en el diseño, cuyo objetivo es realizar una evaluación detallada y completa del diseño del software para identificar y mitigar deficiencias lo antes posible. Esta revisión temprana asegura que las decisiones de diseño se basen en un enfoque de seguridad sólido, minimizando la necesidad de modificaciones costosas en fases posteriores. Al integrar la seguridad en esta fase, el equipo de desarrollo puede avanzar hacia la implementación con una base robusta, reduciendo los riesgos y garantizando un producto final más seguro y confiable.

7.2.1 Principios del diseño seguro de software

El diseño seguro de software se fundamenta en una serie de principios que buscan minimizar las vulnerabilidades y asegurar que las aplicaciones sean robustas frente a ataques. Estos principios no solo protegen los sistemas y los datos, sino que también aseguran la integridad de las operaciones dentro de una organización.

A continuación, se detallan algunos de los principios clave que deben guiar el diseño de software seguro.

Principio de Mínimo Privilegio (Least Privilege)

El principio de mínimo privilegio establece que los usuarios y procesos deben tener únicamente los derechos de acceso mínimos necesarios para realizar sus tareas asignadas. Este enfoque limita la exposición a riesgos, ya que incluso si un usuario o proceso es comprometido, el daño potencial se restringe al nivel de acceso concedido.

Evidencias a solicitar:

- **Historial de Asignación de Privilegios**: registros que muestren cuándo y por quién se asignaron o revocaron los privilegios, junto con la justificación de esos cambios.
- **Registros de Acceso**: logs que muestren qué usuarios acceden a qué sistemas y con qué privilegios, lo que puede ayudar a verificar que los accesos se limitan según el principio de mínimo privilegio.

Principio de Separación de Privilegios (Separation of Privileges)

La separación de privilegios es una técnica de diseño que divide las tareas y privilegios entre diferentes procesos o usuarios para reducir el impacto de una posible brecha de seguridad. Al diferenciar y separar los privilegios necesarios en cada momento de ejecución de un flujo, proceso o programa, se asegura que un atacante no pueda utilizar un solo punto de acceso para realizar acciones no autorizadas o para escalar privilegios dentro del sistema.

Se deberá asegurar que se cumplen con los siguientes requisitos de seguridad:

- Controlar los mecanismos de identificación, autenticación y control de acceso de los desarrolladores, diferenciando rigurosamente los privilegios en cada uno de los entornos existentes.
- En función de cada proyecto y entorno, se deben definir una serie de perfiles de usuario que permita una adecuada segregación de funciones, con el objeto de evitar el acceso a

software, configuración, herramientas e información (como son los documentos de diseño, especificaciones, planes de prueba, etc.) por parte de usuarios no autorizados.

- También se deberá evitar el acceso por parte de los usuarios de desarrollo a los entornos de producción, excepto en aquellos casos que se haya realizado la petición justificada y esta haya sido autorizada por el Responsable de la aplicación designado dentro de la organización.
- Todas las cuentas usadas deben ser nominativas, no genéricas a nombre de un equipo de desarrollo, de forma que pueda realizarse la trazabilidad de los cambios en el código fuente identificando de forma unívoca al desarrollador que los realizó.
- Si la organización lleva a cabo simultáneamente diferentes desarrollos, únicamente deberán acceder a cada proyecto las personas del equipo a quienes se le hubiera asignado el mismo.

Evidencias a solicitar:

- **Políticas de Acceso a Producción o Gestión de cambios**: documentación que especifique los procedimientos para acceder a los entornos de producción, incluyendo ejemplos de justificaciones y autorizaciones.
- **Registros de Cuentas de Usuario**: listado de cuentas de usuario utilizadas por los desarrolladores, mostrando que son nominativas y no genéricas.
- **Registro de Incidentes de Seguridad**: historial de incidentes que involucren violaciones de privilegios y las acciones correctivas tomadas.
- **Documentación de Sistemas de Validación**: detalles sobre los sistemas implementados para validar los accesos a aplicaciones y sistemas, incluyendo cualquier software de monitoreo o gestión de identidades.

Principio de Segregación de Funciones (Segregation of duties –SoD)

La segregación de funciones es un principio de control interno que asegura que ninguna persona o grupo de personas tenga control completo sobre todas las fases de una transacción. Este principio es esencial para prevenir fraudes y errores, garantizando que cada etapa de una transacción esté supervisada por individuos o grupos independientes.

Aplicación en el Diseño de Software:

- **Aspecto Funcional**: una transacción dentro de una aplicación debería dividirse en etapas claras como aprobación, autorización, ejecución y registro. Cada una de estas etapas debe ser gestionada por personas o grupos diferentes para asegurar un control efectivo y reducir la posibilidad de fraude.
- **Aspecto de Gestión**: los roles dentro del ciclo de vida del software, como desarrollo, administración, operación y usuario final, deben estar claramente diferenciados. Por ejemplo, un desarrollador no debería tener acceso a los permisos administrativos del entorno en el que se despliega el software, y los operadores no deberían tener la capacidad de modificar el código fuente.

Evidencias a solicitar:

- **Logs de Acceso y Actividades**: solicitar logs que muestren la actividad de los usuarios dentro de cada fase del desarrollo y operación del software, destacando que los roles están bien diferenciados.
- **Controles de Cambios (Change Control)**: ejemplos de procedimientos de gestión de cambios que aseguren que los desarrolladores no pueden implementar cambios directamente en producción sin revisión y aprobación de otro grupo.

Principio de Mínimos Mecanismos Comunes (Least Common Mechanisms)

El principio de mínimos mecanismos comunes sugiere que los mecanismos que son compartidos por más de un usuario, proceso o función deben minimizarse para reducir la posibilidad de que se utilicen como vectores de ataque. Cuando varios usuarios comparten un mecanismo común, como una variable o un recurso, existe el riesgo de que se convierta en un punto de intercambio de información no controlado.

Principio de Defensa en Profundidad (Defense-in-Depth)

El principio de defensa en profundidad se basa en la implementación de múltiples capas de seguridad para proteger los activos de una organización. La idea es que, si una capa de seguridad es vulnerada, las capas adicionales siguen proporcionando protección.

Principio de Confianza Cero (Zero Trust)

El principio de confianza cero asume que ninguna entidad, ya sea interna o externa, debe ser considerada confiable por defecto. Cada solicitud de acceso debe ser autenticada y autorizada, y se debe mantener una vigilancia continua sobre la actividad para detectar comportamientos anómalos.

Principio de Seguridad en la Apertura (Security-in-the-Open)

El principio de seguridad en la apertura enfatiza la importancia de desarrollar software con seguridad en mente desde el principio, especialmente en el contexto del desarrollo de código abierto. Esto incluye la implementación de prácticas de codificación segura, pruebas rigurosas de vulnerabilidades y la colaboración con expertos en seguridad para garantizar que el código sea seguro.

Fase de implementación

En esta tercera fase de implementación se lleva a cabo el propio desarrollo de software tomando como guía los requisitos y diseño del futuro producto de software, realizados en las dos fases previas.

Durante la implementación el equipo de desarrollo se dedica a la programación del código fuente sobre la plantilla de arquitectura del proyecto o las directrices que tienen que seguir, de esta manera se hace necesaria para el control de seguridad una monitorización de forma periódica de este código y arquitectura; esta revisión es necesaria tanto por parte de los responsables de desarrollo como por parte del auditor de seguridad.

A continuación, se detalla lo necesario para llevar a cabo y documentar correctamente las tareas relacionadas con ciberseguridad para esta fase. Estas revisiones de seguridad estarán compuestas de las siguientes buenas prácticas:

- Revisiones de código dentro del propio equipo de desarrollo, para que al menos más de una persona revise cada nueva introducción o modificación al código fuente. Esta revisión puede ser llevada a cabo sobre el repositorio de código por parte de cada responsable de desarrollo.
- Análisis estático del código mediante herramientas especializadas, preferiblemente de forma automatizada dentro del proceso de integración continua.
- Análisis estático del código manual. Para las partes más críticas del software se hace necesaria una revisión llevada a cabo de manera personalizada y manual por parte de un especialista, para realizar un descubrimiento de vulnerabilidades en mayor profundidad de la que ofrecen las herramientas automáticas.
- Buenas prácticas para el equipo de desarrollo detalladas en el “Procedimiento de desarrollo seguro de aplicaciones”.

7.2.2 Validación de datos de entrada los sistemas

Un punto inicial para revisar buenas prácticas de desarrollo seguro de una aplicación son sus puntos de entrada de datos, para confirmar si estos tienen alguna validación de seguridad. Cada entrada de datos al sistema debe tener una limitación específica en cuanto a cantidad y forma.

Simplificando lo anterior, si nuestra aplicación en un determinado punto debe recibir del usuario su teléfono, podríamos obtener los siguientes ejemplos de entrada de datos validos:

- 6555555
- 655 55 55
- 0034 655 5555
- +34-655-5555

En los anteriores casos se aprecia unos datos relacionados con la petición de la aplicación. Sin embargo, si se introducen los siguientes datos:

- '`"><\x3Cscript>javascript:alert(1)</script>
- <!ENTITY % xxe SYSTEM "file:///etc/passwd">
- " OR 1 = 1 -- -

Puede apreciarse la introducción de unos datos sin relación con la petición de la aplicación y además aparentemente maliciosos, se trata de comandos que buscan atacar la aplicación. Es por este potencial uso malicioso, que la aplicación y su código deben controlar la validación de estos datos.

Para validar los datos de entrada y controlar que estos no se introduzcan al sistema, tenemos varias herramientas disponibles, desde la aplicación de una expresión regular sobre estos, uso de listas blancas, listas negras, parametrización de variables mediante métodos disponibles en librerías, etc.

Para proteger la aplicación no hace falta entender en profundidad el tipo de ataque que puede realizarse, puesto que pueden ser de diversos tipos y muy complejos. El ejemplo más sencillo posible de lo anterior es aplicar una expresión regular al campo de datos “teléfono”:

```
using System;
using System.Text.RegularExpressions;

public static class PhoneNumber
{
  // Regular expression used to validate a phone number.
  public const string motif = @"^([\+]?33[-]?|[0])?[1-9][0-9]{8}$";

  public static bool IsPhoneNbr(string number)
  {
    if (number != null) return Regex.IsMatch(number, motif);
    else return false;
  }
}

public class Program {
  public static void Main(string[] args) {
    Console.WriteLine(PhoneNumber.IsPhoneNbr("+34915977000"));
  }
}
```

Se deben considerar los siguientes aspectos:

- Analizar todos los datos de entrada a las aplicaciones para verificar valores fuera de rango, errores de formato, datos incompletos, volúmenes de datos que excedan los límites inferior y superior, etc. Únicamente se aceptarán datos considerados válidos.
- Filtrar los caracteres inválidos, como los siguientes: ' " / \ * - < > | ; ? & % = ! $, a menos que sean necesarios para la aplicación.
- Disponer de un procedimiento para responder a errores de validación.
- Disponer de un procedimiento para determinar la verosimilitud de los datos.
- Alimentar un registro de logs en base a la validación de los datos de entrada.

Evidencias a solicitar:

- **Documentación detallada sobre los procedimientos de validación de datos, incluyendo**: especificaciones de formato para datos de entrada (por ejemplo, expresiones regulares para números de teléfono, correos electrónicos, etc.).
- **Documentación detallada sobre los procedimientos de validación de datos, incluyendo**: listas blancas y negras de entradas permitidas y prohibidas.
- **Evidencia**: informes de auditorías externas o pruebas de penetración que aborden la validación de datos.

7.2.3 Codificación de salida acorde a la capa de presentación

De igual manera que en el apartado anterior hemos visto cómo controlar que datos maliciosos no pasen a ser procesados por la parte interna del sistema, para evitar ataques e incidentes de seguridad, es igual de importante para la seguridad del sistema que los datos ya preservados en él se representen de forma segura. La forma segura de manejar los datos con los que trabaja la aplicación es codificándolos para su exclusiva representación, de tal manera que las cadenas de caracteres especiales que pudieran llegar a conformar comandos o ejecutar procesos en un sistema queden neutralizados.

Siguiendo un simple ejemplo, si una aplicación web va a presentarle al usuario los datos de una ficha que contienen un campo “comentario”, el cual debe ser permisivo y admitir toda clase de caracteres, nunca le presentará en la página web el campo comentario tal cual, puesto que este podría contener un tag HTML script con código JavaScript en su interior y este ejecutarse en el navegador del citado usuario. A continuación, un ejemplo de texto que al presentarlo sobre nuestra web podría convertirse en código funcional:

- <script>alert('XSS')</script>

En lugar de eso los caracteres serán escapados para su representación, por ejemplo:

- < es convertido a <
- > es convertido a >
- & es convertido a &
- '' es convertido a "

Haciendo uso de esta codificación, la presentación de la cadena de texto quedaría como sigue:

- <script>alert('XSS')</script>

De esta forma los caracteres especiales y reservados en HTML se reemplazan por sus entidades HTML (HTML Entities).

Evidencias a solicitar:

- Captura de pantalla de un resultado de prueba de seguridad donde se verifica la resistencia a ataques XSS.
- Captura de pantalla de la interfaz de usuario donde se presenta un dato con caracteres especiales escapados.

7.2.4 Exposición de información

Un punto importante de cara a no exponer información sensible o interna a un potencial atacante o personal no autorizado es diferenciar un entorno interno y de limitado acceso como puede ser desarrollo, a uno expuesto a los usuarios o incluso al público por internet. En el primero puede ser necesario, como parte del trabajo, exponer los descriptores de los servicios web a consumir, detallando los métodos y parámetros. Los mensajes de error proporcionados deben estar sujetos a reglas. Se deben considerar los siguientes controles:

- Comprobar que los datos de salida son razonables o se ajustan a los rangos esperados.

- En aquellos casos posibles, realizar controles de conciliación para asegurar el correcto procesamiento de todos los datos.
- Alimentar un registro de logs involucradas en los procesos clave respecto a los datos de salida.
- Verificar que la información suministrada por los diferentes errores de procesamiento no proporciona información que pueda ser útil para un posible atacante.
- Los mensajes de error generados no deben revelar información que facilite ataques al sistema, por lo que deben evitarse:
    - Mensajes de error de la aplicación: que contienen textos suficientemente descriptivos que permiten comprometer la plataforma, como mensajes que contienen cadenas de conexión a BBDD, o que contienen rutas de directorios, etc.
    - Mensajes de errores del usuario: deben evitarse mensajes de error que permiten determinar con exactitud qué valor de los diferentes introducidos ha sido el incorrecto, por ejemplo, si corresponden a un proceso de autenticación.

Evidencias a solicitar:

- Captura de pantalla del código fuente que ilustre cómo se gestionan los errores en la aplicación.
- Capturas de pantalla o registros que muestren mensajes de error generados por la aplicación en diferentes escenarios.

7.2.5 Gestión de autenticación y contraseñas

La autenticación es el proceso de verificar que una persona, organización o sitio web es quien dice ser. La autenticación en el contexto de las aplicaciones web se realiza comúnmente enviando un nombre de usuario o código identificativo y uno o más elementos de información privada que solo un usuario determinado debe conocer.

Deber revisarse que los nombres de usuario o códigos identificativos de usuario no distingan entre mayúsculas y minúsculas. El usuario 'Juan' y el usuario 'juan' deben ser el mismo usuario en la práctica, siendo los nombres de usuario en todo caso únicos. Para aplicaciones de alta seguridad, los nombres de usuario podrían asignarse de forma secreta en lugar de usar datos predecibles o públicos como nombre y apellidos.

Es de suma importancia que los usuarios del sistema con permisos elevados, como son los de administración sobre activos, dispongan en el requerimiento de control de acceso al sistema varios factores de autenticación que confirme su identidad. Se recomienda para tal fin usar una contraseña que cumpla las políticas y normas de seguridad de la Organización y el envío de un código mediante SMS, por ejemplo, como segundo factor de autenticación. Tanto para el escenario de usuarios de aplicaciones que ofrecen servicio, como para el escenario de usuarios del sistema como pueden ser de un active directory, todo usuario con alto privilegio debe tener activado el segundo factor de autenticación.

Además, se debe establecer un control de seguridad mediante el cual, se compruebe la complejidad de las contraseñas, impidiendo establecer contraseñas inseguras. En línea con los requerimientos normativos, por ejemplo, en el control del "NIST 500-53 PR.AC-1.CO1" la política de contraseñas debe incluir los siguientes criterios:

- Complejidad.
- Longitud.
- Duración.
- Repetibilidad (no utilización de las últimas contraseñas utilizadas).
- Número de intentos erróneos antes de bloqueo de usuario.

Aquí se muestra una propuesta de control de complejidad de contraseñas:

- Longitud mínima 12 caracteres.
- Requerir al menos un carácter en minúscula.
- Requerir al menos un carácter en mayúscula.
- Requerir al menos un número.
- Requerir al menos un símbolo.
- Tiempo de validez de contraseña 90 días.
- Requisito, no haberse usado la misma contraseña, en las tres veces anteriores que se ha establecido nueva contraseña.

Evidencias a solicitar:

- Captura de pantalla del código que gestiona la creación y validación de nombres de usuario.
- Captura de pantalla que muestre cómo se solicita y valida un segundo factor de autenticación.
- Captura de pantalla de la configuración del sistema que muestra la política de complejidad de contraseñas.
- Captura de pantalla de la configuración que muestra cómo se gestionan los intentos de autenticación fallidos.

7.2.6 Gestión de sesiones

La gestión de sesión deberá tener en cuenta su ciclo de vida (caducidad o duración de esta), para la cual se propone sea configurada de la siguiente manera de acuerdo a la referencia de OWASP sobre la gestión de sesiones:

- 5 minutos para los usuarios que tienen posibilidad de ejecutar funcionalidades consideradas críticas.
- 30 minutos para el resto de los usuarios.

Se deberán usar tokens de sesión adecuados, ya sea usándose en las llamadas web, tanto estableciéndolo como cabecera HTTP como cookie de sesión. Estos tokens deberán tener una complejidad criptográfica suficiente, siendo no inferible (no pudiéndose predecir ni calcular el siguiente token para un usuario). Para este cometido, en cada caso de la plataforma que se esté usando, el framework empleado suele tener una forma nativa de manejar estos aspectos. Desde el punto de vista de seguridad lo mejor es usar la manera más ampliamente contrastada por la industria como son los JWT (Json Web Tokens).

Se debe implementar una funcionalidad que cierre todas las sesiones abiertas para un usuario. Esta funcionalidad debe estar siempre fácilmente visible y accesible por los usuarios, para concienciar y facilitar el uso de esta. Así mismo esta funcionalidad se llamará en el proceso de cambio de credenciales, por ejemplo, invalidando en ese momento del cambio a las nuevas credenciales, cualquier sesión anterior al cambio.

Evidencias a solicitar:

- Captura de pantalla de la configuración del sistema que muestre cómo se establece la duración de las sesiones.
- Captura de pantalla del código que muestra cómo se generan y gestionan los tokens de sesión.
- Captura de pantalla del código o documento que muestre cómo se manejan las sesiones durante el proceso de cambio de credenciales.

7.2.7 Seguridad de base de datos

Para evitar un ataque de tipo inyección SQL, aparte de establecer un control de validación de datos de entrada como se especifica en el primer punto, siempre usaremos consultas a base de datos parametrizadas mediante objetos que protejan de este tipo de ataque.

A continuación se muestra un ejemplo de un código vulnerable, el cual concatena cadenas de texto para conformar la futura consulta SQL a base de datos. En este tipo de código se pueden inyectar comandos maliciosos al concatenarse nuestra posible entrada de datos maliciosa al resto de la consulta SQL:

```
public void Foo(DbContext context, string query, string param)
{
    string sensitiveQuery = string.Concat(query, param);
    context.Database.ExecuteSqlCommand(sensitiveQuery); // Sensitive
    context.Query<User>().FromSql(sensitiveQuery); // Sensitive

    context.Database.ExecuteSqlCommand($"SELECT * FROM mytable WHERE mycol={value}", param); // Sensitive,
the FormattableString is evaluated and converted to RawSqlString
    string query = $"SELECT * FROM mytable WHERE mycol={param}";
    context.Database.ExecuteSqlCommand(query); // Sensitive, the FormattableString has already been evaluated, it
won't be converted to a parametrized query.
}

public void Bar(SqlConnection connection, string param)
```

```
{
   SqlCommand command;
   string sensitiveQuery = string.Format("INSERT INTO Users (name) VALUES (\"{0}\")", param);
   command = new SqlCommand(sensitiveQuery); // Sensitive

   command.CommandText = sensitiveQuery; // Sensitive

   SqlDataAdapter adapter;
   adapter = new SqlDataAdapter(sensitiveQuery, connection); // Sensitive
}
```

A continuación un ejemplo de código seguro, que hace uso de los objetos dispuestos a tal fin (método “ExecuteSqlCommand”) para parametrizar cada uno de los valores a introducir en la consulta que interactúa con la base de datos. De esta manera cada entrada de datos se procesa como un parámetro SQL concretamente, lo que resulta en escapar cualquier carácter especial y tratarlo de manera literal, en lugar de eventualmente ejecutarlo:

```
public void Foo(DbContext context, string query, string param)
{
   context.Database.ExecuteSqlCommand("SELECT * FROM mytable WHERE mycol=@p0", param); // Compliant, it's a
parametrized safe query
}
```

Evidencias a solicitar:

- Capturas de pantalla o registros que demuestren auditorías de las consultas SQL realizadas en la aplicación.
- Informes de herramientas de escaneo de seguridad utilizadas para detectar vulnerabilidades de inyección SQL en el código.

7.2.8 Asegurar suficiente complejidad (robustez) en los procesos criptográficos

Deben usarse los siguientes algoritmos considerados seguros gracias a su complejidad y ausencia de vulnerabilidades conocidas si es necesario:

- SHA-512
- AES (también conocido como Rijndael)
- RSA

Así mismo es importante revisar si en la implementación de estos algoritmos se está usando una longitud de clave suficiente. Se recomienda usar AES con tamaño de clave no menor a 256 y RSA con tamaño de clave no menor a 2048. También es importante revisar la configuración de modo de operación de cifrado por bloques, usar CBC o superior, en lugar de usar el más simpe y desfasado modo ECB, eligiendo una opción de padding segura como es PKCS7, evitando otras más simples y desfasadas como “ZeroBytePadding”.

Excepto indicaciones contrarias, no deben de usarse algoritmos que no vengan incluidos en la lista anterior.

A continuación, una lista con ejemplos comunes de algoritmos que no deben ser implementados a causa de no ofrecer garantía de seguridad, por su falta de complejidad y registro de vulnerabilidades demostradas:

- MD2, MD4, MD5, MD6, HAVAL-128, HMAC-MD5, DSA (que usa SHA-1), RIPEMD, RIPEMD-128, RIPEMD-160, HMACRIPEMD160 y SHA-1

Estos ya no se consideran seguros, porque es posible llegar a producir colisiones en ellos, con mayor o menor esfuerzo computacional.

A continuación, un ejemplo de mala práctica, en el que, para asegurar la futura integridad de un documento de nuestro sistema, tomamos el producto de una función hash para guardar registro de su estado y poder comprobarlo en el futuro. La mala práctica residiría únicamente en la elección de un algoritmo obsoleto como es MD5:

```
public static string MD5Hash(string input)
{
    using (var md5 = MD5.Create())
    {
        var result = md5.ComputeHash(Encoding.ASCII.GetBytes(input));
        return Encoding.ASCII.GetString(result);
    }
}
```

A continuación, una implementación que con el mismo diseño, elige un algoritmo más actual y válido:

```
using System;
using System.Text;
using System.Security.Cryptography;

public class Example
{
    static string ComputeSHA512(string s)
    {
        StringBuilder sb = new StringBuilder();
        using (SHA512 sha512 = SHA512.Create())
        {
            byte[] hashValue = sha512.ComputeHash(Encoding.UTF8.GetBytes(s));
            foreach (byte b in hashValue) {
                sb.Append($"{b:X2}");
            }
        }

        return sb.ToString();
    }

    public static void Main()
    {
        string hashValue = ComputeSHA512("Some string");
        Console.WriteLine(hashValue);
    }
```

```
}
```

Evidencias a solicitar:

- **Política de Cifrado**: debe detallar los algoritmos permitidos (e.g., SHA-512, AES-256, RSA-2048) y las configuraciones recomendadas (por ejemplo, modos CBC, padding PKCS7, etc.).
- **Procedimientos de Gestión de Claves**: documentación sobre generación, almacenamiento, rotación y destrucción de claves criptográficas.

7.2.9 Comunicación segura

El primer paso para dar una garantía de seguridad en la comunicación es impedir o avisar de que una conexión no segura, es decir HTTP en texto plano, pueda ser posible. Para ello el sistema no debe de poder ofrecer este tipo de conectividad bajo ningún contexto, para que nadie pueda conectarse de esta manera ni siquiera por error.

En segundo lugar, una vez que todas nuestras comunicaciones se establezcan de forma segura, daremos posibilidad de que esto suceda solo y exclusivamente desde unos algoritmos criptográficos actuales y adecuados, impidiendo el uso de algoritmos que expongan nuestra seguridad y datos por antiguos y obsoletos, ya que en estos casos pueden prepararse ataques específicos.

Por otro lado, se recomienda deshabilitar los protocolos de comunicación segura que han quedado desfasados y débiles, usando por lo tanto solo protocolo TLS 1.2 o superior. Los intentos de acceso web por el puerto 80 deben redirigirse al puerto 443.

Evidencias a solicitar:

- **Listas de Cifrado Permitidas:** configuraciones que incluyan únicamente algoritmos seguros como AES-256, SHA-512, RSA-2048 o superiores.
- **Reglas de Redirección HTTP a HTTPS:** capturas de pantalla o fragmentos de código que demuestren las reglas de redirección.

7.2.10 Gestión de ficheros

Cuando el código trabaja con el sistema de archivos, la principal vulnerabilidad a mitigar es el conocido como “Path Trasversal”. Este tipo de vulnerabilidad consiste en engañar al sistema mediante uso de caracteres especiales en la ruta del archivo.

El caso más típico es el siguiente, nuestra aplicación debe devolver el contenido de un archivo al usuario y el nombre de este archivo es especificado por el propio usuario o está definido desde la capa cliente, por lo que si desde el front-end nos llega este valor “documento1.pdf”, nuestra aplicación le devolverá el contenido de este archivo PDF al usuario. El ataque consiste en que el usuario especifica en el nombre de archivo otro archivo al que no debería de poder acceder por sus permisos y además haciendo uso de caracteres especiales de la ruta del sistema de archivos, por ejemplo:

```
../../etc/passwd
```

Si el ataque tiene éxito, nuestra aplicación pretenderá abrir una carpeta concreta, por ejemplo "/opt/AppPdfDocuments", sin embargo al conformarse con lo suministrado desde el front-end finalmente genera la cadena de texto "/opt/AppPdfDocuments/../../etc/passwd". Los dobles puntos nos llevarían a la raíz del servidor y posteriormente devolveríamos el contenido de un archivo reservado del sistema, cuando deberíamos devolver únicamente archivos contenidos en la carpeta "AppPdfDocuments".

Para evitar esto debemos establecer un control y una limitación en la gestión de la cadena de texto que conformará la futura ruta con la que nuestro código trabajará con el sistema de archivos. A continuación, se muestra un código vulnerable, que no controla la seguridad de los datos de entrada:

```
public class ExampleController : Controller
{
   private static string TargetDirectory = "/path/to/target/directory/";

   public void Example(string filename)
   {
      string path = Path.Combine(TargetDirectory, filename);
      System.IO.File.Delete(path);
   }
}
```

A continuación, el mismo fragmento de código pero con un control de seguridad sobre los datos de entrada y por lo tanto sobre el posterior trabajo con el sistema de archivos:

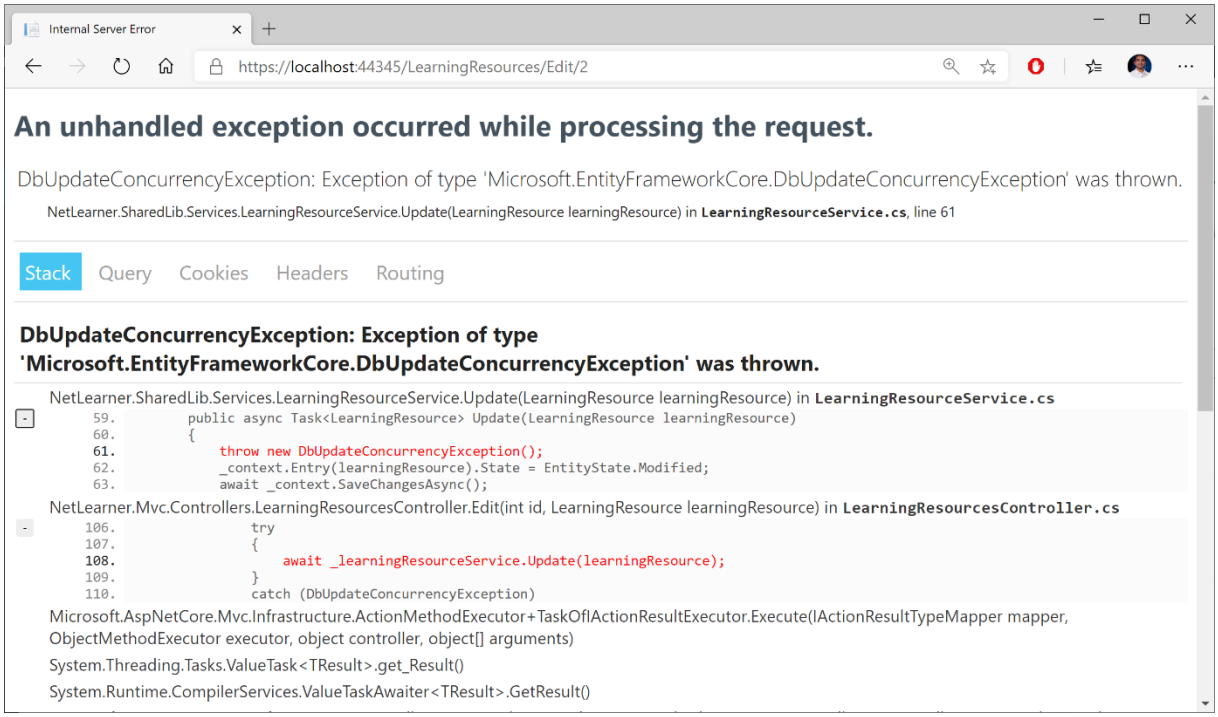
```
public class ExampleController : Controller
{
   private static string TargetDirectory = "/path/to/target/directory/";

   public void Example(string filename)
   {
      string path = Path.Combine(TargetDirectory, filename);
      string canonicalDestinationPath = Path.GetFullPath(path);

      if (canonicalDestinationPath.StartsWith(TargetDirectory, StringComparison.Ordinal))
      {
         System.IO.File.Delete(canonicalDestinationPath);
      }
   }
}
```

7.2.11 Manejo y registro de errores

En lo relativo a la seguridad es muy importante que la aplicación no "regale" información relacionada con errores a un potencial atacante. Incluso cuando una aplicación no tiene conocida una vulnerabilidad concreta que explotar, una mala gestión de errores se convierte en un problema. Cuando una aplicación tiene malas prácticas en su codificación, en lo relativo a la gestión de errores, podrás apreciar que se comporta de forma inestable y ofrece respuestas que están fuera de lugar al usuario, como por ejemplo una pantalla de error, con un error muy técnico e interno al servidor que ofrece la aplicación. Este tipo de mala práctica indica a un potencial atacante que las cosas no

están bien hechas y puede provocar una investigación más profunda para descubrir vulnerabilidades.

A continuación, mostramos un ejemplo en el que una aplicación expone un error interno del servidor a un usuario que no va a poder hacer nada con esta información, esto muestra debilidad por parte del sistema.



Este tipo de páginas únicamente deben ser mostradas en un entorno de desarrollo donde se trabaje de forma interna con la aplicación y por lo tanto esta información técnica ayude a personal autorizado a identificar y corregir errores. En ningún caso la aplicación debe estar configurada de esta manera cuando está expuesta a usuarios.

Una excepción es un evento inusual que altera la ejecución esperada del software. La buena práctica en este punto es contemplar y procesar estas condiciones excepcionales de una manera adecuada.

El código que sigue buenas prácticas en cuanto a manejo de errores, está estructurado de forma que encapsula cada parte de código susceptible de excepciones y recoge estas con código destinado a tratar estas de la forma más específica posible, fallando de forma segura y controlada, registrando lo ocurrido y realizando unas últimas acciones necesarias, como liberando o devolviendo a un estado previo un recurso. Según lo ocurrido y el contexto del software se diseñará su respuesta, puede llegarse incluso a bloquear o detener el servicio de aplicación hasta la revisión que confirme el estado de esta, para evitar males mayores como una destrucción de datos. A continuación se muestra un ejemplo en entorno tecnológico C#:

```
WebClient wc = null;
try
{
```

```
  wc = new WebClient(); //downloading a web page
  var resultData = wc.DownloadString("http://google.com");
}
catch (ArgumentNullException ex)
{
  // Manejar este tipo de exception concreta, que implicacion tiene y como respondemos?
}
catch (WebException ex)
{
  // Manejar este tipo de exception concreta, que implicacion tiene y como respondemos?
}
catch (Exception ex)
{
  // Ha ocurrido una exception que no contemplabamos dentro de nuestra planificacion inicial, daremos la respuesta por
defecto y registraremos lo sucedido
}
finally
{
  // Este código será ejecutado en cualquier caso
  // Para este ejemplo, hacemos un dispose del objeto WebClient
  wc?.Dispose();
}
```

7.2.12 Gestión de memoria

La gestión de memoria es una de las áreas más importantes de la seguridad del software. Los errores de gestión de memoria pueden conducir a vulnerabilidades de seguridad que pueden ser explotadas por los atacantes para tomar el control del sistema o robar datos. A continuación, veremos algunas medidas de seguridad que los desarrolladores pueden tomar para proteger su software de los errores de gestión de memoria:

- **Usar un lenguaje de programación seguro:** algunos lenguajes de programación, como C y C++, son más propensos a errores de gestión de memoria que otros lenguajes, como Java y Python. Esto es importante a tener en cuenta para elegir preferiblemente un lenguaje de programación que sea seguro por el propio diseño.
- **Utilizar herramientas de depuración:** las herramientas de depuración pueden ayudar a los desarrolladores a identificar y corregir los errores de gestión de memoria.
- **Escribir código seguro:** los desarrolladores deben seguir las buenas prácticas de codificación para evitar los errores de gestión de memoria. Estas prácticas incluyen:
  - Usar los objetos y métodos adecuados en ese entorno, para asignar memoria de forma segura.
  - Liberar la memoria que ya no se necesita.
  - Evitar el uso de punteros a objetos inválidos.
  - Probar el código exhaustivamente.

Aquí hay algunos ejemplos específicos de errores/vulnerabilidades en la gestión de memoria y cómo evitarlos:

- **Desbordamiento de búfer:** esto ocurre cuando se escribe más datos en un búfer de lo que puede contener. Esto puede hacer que el software se bloquee o se comporte de forma inesperada. Para evitar los desbordamientos de búfer los desarrolladores deben asegurarse de que los búferes sean lo suficientemente grandes para los datos que se van a almacenar.
- **Fuga de memoria:** esto ocurre cuando el software no libera la memoria que ya no se necesita. Esto puede hacer que el sistema se quede sin memoria y se bloquee. Para evitar las fugas de memoria los desarrolladores deben asegurarse de liberar la memoria que ya no se necesita.
- **Apuntador a objeto inválido:** esto ocurre cuando un puntero apunta a un objeto que ya no existe. Esto puede hacer que el software se comporte de forma inesperada o se bloquee. Para evitar los punteros a objetos inválidos los desarrolladores deben asegurarse de que los punteros siempre apunten a objetos válidos.

1.1.1. Prácticas generales de codificación

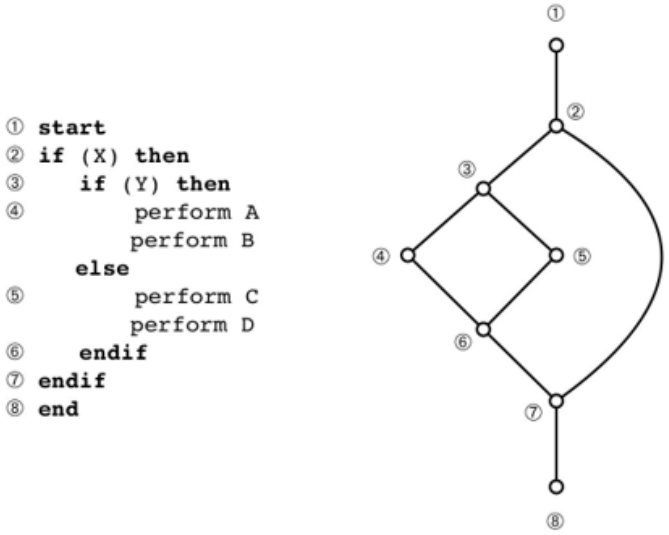
Como practicas generales de codificación a seguir es importante tener en cuenta los principios de diseño de software “SOLID”, que ayudan a crear software más sólido, mantenible y escalable. Los principios son:

- **Principio de responsabilidad única (SRP):** cada módulo, clase o función debe tener una sola responsabilidad. Esto hace que el código sea más fácil de entender, probar y mantener.
- **Principio de abierto/cerrado (OCP):** el software debe estar abierto a la extensión, pero cerrado a la modificación. Esto hace que el software sea más fácil de mantener y actualizar.
- **Principio de sustitución de Liskov (LSP):** las clases hijas deben poder sustituir a sus clases padres sin romper el código. Esto hace que el software sea más flexible y adaptable.
- **Principio de segregación de interfaces (ISP):** las interfaces deben ser tan pequeñas y específicas como sea posible. Esto hace que el software sea más modular y flexible.
- **Principio de inversión de dependencias (DIP):** las clases de alto nivel no deben depender de clases de bajo nivel. En cambio, ambas clases deben depender de abstracciones. Esto hace que el software sea más flexible y fácil de mantener.

Estos principios pueden ser aplicados a cualquier lenguaje de programación, pero son especialmente relevantes para el desarrollo de software orientado a objetos.

Asimismo, es importante conservar una baja complejidad ciclomática en el código. La complejidad ciclomática es una métrica de software que mide la complejidad lógica de un programa o sección de código, basada en el número de caminos independientes que hay en el código. Un camino independiente es una secuencia de instrucciones que se puede ejecutar sin repetir ninguna instrucción.

La complejidad ciclomática es una medida de la dificultad de entender, probar y mantener un código. Los códigos con una complejidad ciclomática alta son más difíciles de entender, probar y mantener ya que hay más caminos independientes que hay que seguir para entender cómo funciona el código. En general, se recomienda que la complejidad ciclomática de un código no sea superior a 10. Sin embargo, este umbral puede variar en función del lenguaje de programación y de la complejidad del código.



Fase de pruebas

En esta fase el producto software se encuentra lo suficientemente desarrollado como para que el equipo de control de calidad pueda llevar a cabo pruebas sobre él, desde las pruebas iniciales unitarias, pasando por pruebas básicas de integración, hasta pruebas finales de una reléase concreta del producto.

Además del control de calidad, cuya necesidad es mucho más fácil de intuir por los participantes en el desarrollo del producto software, es igual de importante el control de seguridad durante esta fase, por ello en este documento se indica como comenzar a llevar a cabo pruebas de seguridad, tanto dinámicas como estáticas.

A continuación, se detalla lo necesario para llevar a cabo y documentar correctamente, las tareas relacionadas con Ciberseguridad para esta fase. Estas revisiones de seguridad estarán compuestas de las siguientes buenas prácticas:

- **Pruebas dinámicas de seguridad (DAST)** sobre el componente software: consiste en llevar a cabo un descubrimiento de vulnerabilidades y una evaluación de potenciales debilidades del producto software, desde la realización de pruebas sobre un entorno de ejecución de dicho software.
- **Pruebas estáticas de seguridad (SAST)** sobre el componente software: consiste en llevar a cabo un descubrimiento de vulnerabilidades y una evaluación de potenciales debilidades del producto software, desde la realización de revisiones de su código.
- **No dejar de lado los objetivos generales**: como por ejemplo revisar y confirmar que los requisitos de seguridad establecidos en la primera fase se están cumpliendo en una forma correcta. De no confirmarse la seguridad del producto software de forma completa, es muy importante trasladar estos riesgos a la dirección del proyecto.

Fase de mantenimiento

En esta fase el producto software ya se encuentra desarrollado por completo, por lo tanto, los aspectos principales a controlar desde el punto de vista de la seguridad dejan de tener implicación en la construcción del software, para pasan a centrarse en la distribución y el mantenimiento del software ya terminado.

Diferenciaremos las tareas que conlleva esta fase en dos partes:

- **Distribución del software**: el despliegue de la aplicación por parte del personal de administración de sistemas debe disponer de una serie de comprobaciones de seguridad documentadas que puedan seguir. No solo importa llevar a cabo una correcta y completa configuración y despliegue de la aplicación objetivo, si no también revisar la seguridad del propio servidor y plataforma base en la que esta se despliega. En este sentido debe realizarse una fortificación del activo en general.

  A continuación, un ejemplo de puntos que deben repasarse durante la distribución:

  - Cambiar contraseñas y claves de entornos anteriores, adecuada gestión de claves.
  - Cambiar configuración que venga por defecto y sea insegura.
  - Eliminar o deshabilitar toda la funcionalidad no requerida en nuestro despliegue.
  - Seguir la guía de fortificación adecuada para ese componente.

- **Mantenimiento del software**: a lo largo de la vida útil del software se requerirá de una monitorización continua relativa a la seguridad. Esto implica unas auditorías de seguridad periódicas y la atención de un equipo de respuesta a ciberincidentes para los potenciales incidentes de seguridad que surjan. En este sentido se conducirán de forma periódica auditorías de seguridad conformadas con ejecución de herramientas automáticas y también pruebas dinámicas manuales.

  A continuación, un ejemplo de tareas generales a la gestión de la seguridad, de las que debe disponer la organización:

  - Monitorización continua con descubrimiento periódico de vulnerabilidades.
  - Equipo de Respuesta a Ciberincidentes (ERC).

8. ACEPTACIÓN Y PUESTA EN SERVICIO

La aceptación y puesta en servicio de una aplicación es un proceso fundamental para garantizar su seguridad. En esta fase, se debe comprobar que la aplicación cumple los requisitos de seguridad establecidos y que no introduce vulnerabilidades que puedan poner en riesgo la información o los sistemas de la organización.

El texto que se presenta a continuación describe los requisitos de seguridad que deben cumplirse durante la aceptación y puesta en servicio de una aplicación. Estos requisitos se basan en las recomendaciones de la Guía de Seguridad de las TIC del Centro Criptológico Nacional (ENS).

Estos requisitos incluyen la comprobación del correcto funcionamiento de la aplicación, el cumplimiento de los criterios de aceptación en materia de seguridad y la comprobación de que la aplicación no deteriora la seguridad de otros componentes del servicio. Además, se describe el requisito de aportación de suficientes evidencias de la seguridad de la aplicación por parte del proveedor, la disponibilidad de guías de instalación y configuración segura, uso seguro y relación entre cliente y proveedor, y la realización de pruebas en un entorno aislado de preproducción y auditorías de código fuente.

Evidencias a entregar y aprobación del responsable

Las evidencias y verificaciones que garantizan los criterios de aceptación en materia de seguridad, propuestos por la guía CCN-STIC 808 del Esquema Nacional de Seguridad (ENS), son los siguientes:

1. Datos e informes sobre las pruebas de seguridad realizadas al componente software:
    a. Resultados y respuesta a las pruebas dinámicas (DAST o pentest).
    b. Resultados y respuesta a las pruebas estáticas (SAST o auditoría de código).
2. Los siguientes documentos (los que procedan dada la naturaleza del componente software) facilitados por el equipo de desarrollo interno o el proveedor externo:
    a. Guías de instalación y configuración segura del sistema.
    b. Guías de uso seguro del sistema.
    c. Guías de relación entre cliente y proveedor.
3. Evidencia de la existencia y uso del entorno de pruebas o preproducción (entorno donde realizar las pruebas de forma segura).
4. Evidencia de una verificación de integridad, respecto a la seguridad de otras aplicaciones y/o elementos del sistema.

Los siguientes puntos de este documento, describen cada uno de los entregables anteriores y la necesidad que los motiva.

Para la puesta en producción de un nuevo componente del sistema y para sus sucesivas actualizaciones, se deberá de contar con la correspondiente autorización por parte del Comité de Cambios, tras la cual se procederá a gestionar el cambio en el sistema.

Para la autorización de la puesta en producción de un componente o servicio TIC, se seguirán las pautas establecidas en el "**PROCEDIMIENTO DE GESTIÓN DE CAMBIOS"** de la Organización.

Pasos del procedimiento de aceptación y puesta en servicio:

Pruebas de seguridad a realizar al componente software

Las pruebas de seguridad son un proceso fundamental para garantizar la seguridad de los sistemas software. Estas pruebas tienen como objetivo identificar y corregir vulnerabilidades que podrían ser explotadas por atacantes para comprometer la seguridad del sistema. Es por ello que se requiere

que las correcciones a las vulnerabilidades encontradas sean realizadas y aportadas como evidencias al proceso.

Las pruebas de seguridad se pueden dividir en dos categorías principales: pruebas dinámicas y pruebas estáticas. Normalmente se entregarán al responsable del contrato los resultados de estas pruebas en dos informes diferentes o en dos apartados diferenciados dentro de un único informe:

- **Pruebas dinámicas (DAST o pentest)**: estas pruebas se centran en la detección de vulnerabilidades que pueden ser explotadas durante la ejecución del software. Las pruebas dinámicas pueden incluir pruebas de caja negra, pruebas de caja blanca y pruebas de caja gris.

  En un sistema de información con superficie de exposición a internet la seguridad no solo depende del software, sino del conjunto hardware/software. En este sentido, las pruebas de penetración o pentesting deben realizarse sobre la infraestructura en producción, de modo que intervengan todos los elementos del sistema involucrados.

- **Pruebas estáticas (SAST o auditoría de código)**: las pruebas estáticas se realizan analizando el código fuente del software. Estas pruebas se centran en la detección de vulnerabilidades de forma interna, desde la lectura del código y la propia revisión de la arquitectura del componente software.

Los informes de pruebas de seguridad deben ser claros, concisos y fáciles de entender. Los informes incluirán la siguiente información:

- Descripción de cada una de las vulnerabilidades que se encontraron.
- Descripción de cómo se pueden corregir cada una de estas vulnerabilidades.

La respuesta a estos resultados de seguridad por parte del equipo de desarrollo de software debe igualmente ser clara, concisa y fácil de entender para cada vulnerabilidad, incluyendo la siguiente información:

- Respuesta dada a la vulnerabilidad y corrección descrita en el informe.
- Confirmación de que esta respuesta es suficiente.

Las vulnerabilidades encontradas deben ser corregidas lo antes posible. Las correcciones de vulnerabilidades pueden incluir parches de software, actualizaciones de código o cambios en los procesos de desarrollo.

Documentación del componente software

La documentación de un componente software debe redactarse para garantizar su correcto funcionamiento y seguridad. Esta documentación ayuda a los usuarios a comprender el componente, instalarlo y configurarlo correctamente, evitando errores, proporcionando información sobre cómo instalar, configurar, usar y mantener el componente de forma segura.

La documentación puede variar en función de la complejidad del componente software y de los requisitos de este, sin embargo, se deben aportar al menos los siguientes documentos:

- **Guías de instalación y configuración segura del sistema**: estas guías proporcionan instrucciones paso a paso sobre cómo instalar y configurar el componente de forma segura.

Incluyen información sobre los requisitos del sistema, las opciones de configuración y los procedimientos de seguridad.

- **Guías de uso seguro del sistema**: estas guías proporcionan información sobre cómo usar el componente de forma segura. Incluyen información sobre las características y funcionalidades del componente, así como las recomendaciones para su uso seguro.
- **Guías de relación entre cliente y proveedor**: estas guías proporcionan información sobre los términos y condiciones del contrato entre el cliente y el proveedor del componente. Incluyen información sobre la responsabilidad de cada parte, los procedimientos comunicación, soporte y los recursos disponibles.

Entorno de pruebas o preproducción

El entorno de pruebas o preproducción es un entorno que ha sido aislado de forma segura para realizar las necesarias pruebas de software antes de su despliegue en producción. Este entorno es absolutamente indispensable para poder probar de manera adecuada que el software funciona correctamente y cumple con los requisitos de calidad y seguridad establecidos.

En este entorno se realizan pruebas de control de calidad y control de seguridad, siendo por lo tanto muy beneficioso en muchos casos preparar dos entornos diferenciados de preproducción, uno para calidad y otro para seguridad. Lo anterior tiene causa en que normalmente el personal que realiza el control de calidad es completamente independiente del personal que realiza las pruebas de seguridad, y en principio no tienen una necesidad de estar coordinados. Asimismo, unas pruebas de seguridad adecuadas y completas potencialmente producen escenarios de denegación de servicio o pérdida de datos al entorno. Esto es beneficioso que se produzca y por lo tanto que se conozca, siempre y cuando el entorno entregado a pruebas de seguridad este aislado y no tenga repercusiones en caso de pérdida de servicio o datos.

El entorno de pruebas o preproducción debe ser idéntico al entorno de producción, salvo diferencias que se hagan estrictamente necesarias, ya que el componente software que hay que probar es el que se va a desplegar en la práctica en producción.

La importancia básica de este requisito hace que se requiera presentar evidencia de su existencia y uso. Esta evidencia bastará con que sea una confirmación por escrito del responsable del proyecto, preferiblemente con capturas de pantalla. La anterior evidencia puede agregarse a la propia documentación del componente software.

Verificación de seguridad en la interrelación del componente software con el resto de las componentes del sistema

La necesidad de elaborar un documento que confirme que se ha verificado por seguridad la interrelación del componente software que se está desarrollando con el resto de los componentes del sistema radica en la importancia de garantizar la seguridad del sistema en su conjunto.

Un sistema informático está compuesto por una serie de componentes que interactúan entre sí, si uno de estos componentes presenta una vulnerabilidad de seguridad puede comprometer la seguridad del sistema en su conjunto. Por lo tanto, es importante verificar la seguridad de la

interrelación de todos los componentes del sistema para garantizar que no existen vulnerabilidades que puedan ser explotadas por un atacante.

Lo importante en este punto es la revisión de seguridad por parte de los colaboradores del proyecto. El documento entregable, que puede ser un apéndice dentro de otro documento del proyecto, sencillamente será una confirmación de que esta revisión se ha realizado y una enumeración en forma de lista con las diferentes interrelaciones que tiene este componente con otros componentes del sistema con una descripción de cada una de estas relaciones.

9. DESARROLLO EXTERNALIZADO

En los contratos de desarrollo ejecutados por terceros, se tendrá en cuenta lo siguiente:

- Deberán establecerse acuerdos sobre licencia, propiedad del código y derechos de propiedad intelectual.
- Deberán establecerse requerimientos sobre metodologías de desarrollo seguro, codificación segura, y metodología de pruebas que tengan en cuenta la seguridad junto a la calidad del código.
- Deberá establecerse un protocolo de revisión de los desarrollos realizados que garantice:
  - La ausencia de contenido malicioso, intencionado o no, a través de la aportación de evidencias.
  - La ausencia de vulnerabilidades conocidas, a través de la aportación de evidencias.
- En los desarrollos de sistemas por parte de terceros se garantizará por contrato la calidad y seguridad del software entregado. A ser posible, se responsabilizará a la tercera parte de los daños en los que la Organización pudiera incurrir en caso contrario.
- En los desarrollos de sistemas por parte de terceros las terceras partes firmarán acuerdos o cláusulas de confidencialidad y no divulgación.
- Se seguirá lo requerido en la **Instrucción relativa al clausulado a incluir en materia de protección de datos, transparencia y seguridad en los pliegos de contratación** para la contratación de un servicio externo de desarrollo de aplicaciones.

10. DOCUMENTACIÓN COMPLEMENTARIA

- Norma de gestión de la documentación.
- Procedimiento de gestión de autorizaciones.
- Procedimiento de aceptación y puesta en servicio.
- Procedimiento de copias de seguridad.
- Guía CCN-STIC-205 - Actividades Seguridad Ciclo Vida CIS.
- Guía CCN-CERT BP/28 Recomendaciones sobre desarrollo seguro.
- Guía CCN-STIC 808 ENS. Verificación del cumplimiento.

11. COMUNICACIÓN DE DEFICIENCIAS DEL PROCEDIMIENTO

Cualquier deficiencia y/o inconsistencia en este procedimiento deberá de ser puesta en conocimiento del Responsable del SGSI.

12. REFERENCIAS

- ISO/ IEC 27001:2022:
  - 8.29 Pruebas de seguridad en desarrollo y aceptación
  - 8.31 Separación de los entornos de desarrollo, prueba y producción
  - 8.32 Gestión del cambio
  - 8.33 Datos de Prueba
  - 8.11 Enmascaramiento de datos
  - 8.30 Externalización del desarrollo
  - 8.27 Arquitectura segura de sistemas y principios de ingeniería
  - 5.8 Seguridad de la información en la gestión de proyectos

ANEXO I. ANÁLISIS DE RIESGOS

NOMBRE DE LA APLICACIÓN:		
1. Descripción general del proyecto		
Funcionalidad		
Tecnologías utilizadas		
Ambiente de despliegue		
2. Identificación de activos críticos	**Descripción de Activos críticos**	
Datos personales de usuarios		
Información financiera		
Acceso a sistemas internos		
3. Amenazas potenciales	**Descripción de la Amenaza**	
SQL inyection		
Cross-Site Scripting (XSS)		
[…]		
4. Vulnerabilidades	**Nivel de Riesgo**	
Uso de bibliotecas desactualizadas		
Falta de validación de entradas		
[…]		
5. Mitigación	**Amenaza**	**Vulnerabilidad**
Forzar uso de HTTPS en toda la aplicación	Robo de credenciales	Sesiones no cifradas
[…]		

ANEXO II. CHECKLIST REQUISITOS DE SEGURIDAD

	CATEGORÍA DEL SISTEMA		
	MEDIA	**ALTA**	**COMPROBACIÓN**
GESTIÓN DE RIESGOS Y AMENAZAS WEB			
Análisis de riesgos.	X	X	
INCIDENTES DE SEGURIDAD EN ENTORNOS WEB			
Sistema de registro de incidentes de seguridad.	X	X	
ESTRATEGIA Y METODOLOGÍAS EN ENTORNOS WEB			
Metodología de Seguridad en el desarrollo de aplicaciones web.	X	X	
El almacenamiento de información sensible, tanto propia de la lógica de la aplicación, debe de almacenarse cifrada en todos los servidores, y especialmente en el de base de datos.		X	
El almacenamiento de las credenciales de acceso deberá realizarse en forma de hash.		X	
ARQUITECTURA DE LA APLICACIÓN WEB			
Deben existir recomendaciones de seguridad asociadas a los dispositivos y entorno de red necesario para el despliegue de la aplicación Web: switches, routers, firewalls, etc.	X	X	
Deben existir recomendaciones de seguridad asociadas a los servidores y sistema operativo de los equipos necesarios para implantar la aplicación Web: servidor Web, servidor de aplicaciones o servidor de base de datos.	X	X	
Deben existir recomendaciones respecto a las actualizaciones de seguridad de los diferentes elementos software qué conforman la plataforma de la aplicación Web: software de los equipos de red, sistema operativo de los servidores, software del entorno Web, etc.	X	X	
Determinar el tipo de arquitectura se emplea en la aplicación Web, dos o tres capas, así como el por qué y los aspectos considerados para aplicar este diseño.	X	X	
Deben existir elementos de detección (sistema de detección de intrusos) y de protección (firewall de aplicación Web, WAF) en la arquitectura. Identificar dónde están situados en la topología de red y la justificación de esa ubicación.	X	X	
Deben existir elementos de detección (sistema de detección de intrusos) y de protección (firewall de aplicación Web, WAF) específicos entre el servidor Web y el de aplicaciones. Verificar si se dispone de estos elementos.	X	X	

	CATEGORÍA DEL SISTEMA		
	MEDIA	**ALTA**	**COMPROBACIÓN**
Deben existir elementos de detección (sistema de detección de intrusos) y de protección (firewall de aplicación Web, WAF) específicos entre el servidor de aplicaciones y el de base de datos. Verificar si se dispone de estos elementos.		**X**	
Identificar qué dispositivo de tipo WAF se emplea y por qué, así como los criterios empleados para su elección.	**X**	**X**	
Se deberá utilizar tráfico cifrado HTTPS en la aplicación web, e identificar si existen dispositivos de seguridad para el análisis de este tipo de tráfico.	**X**	**X**	
Identificar cuál es el servidor Web empleado para la aplicación Web: Apache, IIS, etc., y verificar cuáles son las recomendaciones de seguridad para la instalación y configuración del servidor Web.	**X**	**X**	
Identificar cuál es el servidor de aplicación empleado para la aplicación Web: Tomcat, WebSphere, Weblogic, Jakarta, etc., y verificar cuáles son las recomendaciones de seguridad para la instalación y configuración del servidor de aplicación.	**X**	**X**	
Identificar cuál es el servidor de base de datos empleado para la aplicación Web: MySQL, Microsoft SQL Server, Oracle, DB2, PostgreSQL, etc, y verificar cuáles son las recomendaciones de seguridad para la instalación y configuración del servidor de base de datos.	**X**	**X**	
Comprobar si se emplean aplicaciones Web de terceros para proporcionar parte de la infraestructura básica de la aplicación Web, tal como PHPBB, WordPress, etc. En caso afirmativo, verificar cuáles son las recomendaciones de seguridad para la instalación y configuración del software Web de terceros.	**X**	**X**	
Identificar si se emplea HTTPS para el acceso a todos los recursos confidenciales de la aplicación, tales como interfaces de administración.	**X**	**X**	
Identificar si se emplea HTTPS para todas las transacciones Web que requieren reforzar la confidencialidad de la comunicación, tales como sesiones de usuario con información sensible.	**X**	**X**	
Verificar si se emplea SSL o TLS en algún módulo de la aplicación y el porqué. Comprobar la versión concreta del protocolo empleada.	**X**	**X**	
Verificar si existe algún mecanismo en el diseño o arquitectura de la aplicación para hacer frente a ataques de denegación de servicio (DoS).		**X**	
Verificar si se aplican en la infraestructura de Filtrado, tanto filtros de tráfico de entrada (ingress) como de salida (egress).	**X**	**X**	

	CATEGORÍA DEL SISTEMA		
	MEDIA	**ALTA**	**COMPROBACIÓN**
Identificar si se dispone de información detallada sobre las credenciales y permisos de acceso necesarios para la ejecución de cada componente de software existente en todos los servidores, así como para las interacciones entre éstos.	**X**	**X**	
DESARROLLO SEGURO DE LA APLICACIÓN WEB			
Verificar si existe una metodología para incluir la seguridad como elemento fundamental del ciclo de vida de desarrollo de software como por ejemplo, CLASP de OWASP. Obtener detalles sobre la metodología empleada y las acciones que conlleva.	**X**	**X**	
Identificar el entorno de desarrollo empleado en la aplicación Web: PHP, ASP, ASP .NET, Java (J2EE, JSP…), Python, Ruby on Rails, Perl, etc, así como las recomendaciones de seguridad específicas para la instalación y configuración del entorno de desarrollo.	**X**	**X**	
Identificar el lenguaje de programación empleado en la aplicación Web, en la parte servidor: PHP, ASP, ASP .NET, Java, Python, Ruby, Perl, etc, así como la metodología de seguridad específica empleada en el desarrollo de código mediante el/los lenguajes/s empleados.	**X**	**X**	
Identificar el lenguaje de programación empleado en la aplicación Web, en la parte cliente: AJAX, JavaScript, VBScript, ActiveX, Flash, Java applets, etc, así como la metodología de seguridad específica empleada en el desarrollo de código mediante el/los lenguajes/s empleadoscomo el por qué y los aspectos considerados para aplicar este diseño.	**X**	**X**	
Verificar si el desarrollo contempla la protección frente a las vulnerabilidades Web más comunes.	**X**	**X**	
En caso de respuesta afirmativa en el punto anterior, identificar qué metodología se emplea para desarrollar una aplicación Web segura y protegida frente a las vulnerabilidades más comunes.	**X**	**X**	
Comprobar si existe algún mecanismo de soporte tras el paso a producción de la aplicación que permita obtener actualizaciones software que resuelvan nuevas vulnerabilidades de seguridad encontradas en la aplicación Web.	**X**	**X**	
Identificar qué modelo de filtrado se emplea en la aplicación (eliminación de caracteres maliciosos o aceptación de caracteres válidos únicamente) y los motivos para la aplicación de este modelo.		**X**	
Identificar los mecanismos de filtrado de los datos del usuario empleados por todos los componentes de la aplicación Web que deben interactuar con el cliente.		**X**	

	CATEGORÍA DEL SISTEMA		
	MEDIA	**ALTA**	**COMPROBACIÓN**
Verificar si se normaliza la entrada del usuario antes de proceder a su verificación y filtrado.	**X**	**X**	
Identificar los métodos de normalización contemplados en la aplicación: ASCII, codificación de URLs en hexadecimal, Unicode, etc.	**X**	**X**	
Verificar si se dispone de una librería única y centralizada que proporcione todas las funciones de normalización y filtrado de los datos recibidos por el usuario. Asimismo, identificar para qué tipo de ataques proporciona funciones específicas esta librería: XSS, inyección, referencias a ficheros, etc., así como el conjunto de caracteres que se filtran o aceptan para cada tipo de ataque.		**X**	
Verificar dónde se realiza el filtrado de los datos de entrada: en el cliente, en el servidor o en ambos, así como el motivo por el qué se aplica un método u otro.		**X**	
Verificar si se han implementado mecanismos de protección en la aplicación frente a ataques de CSRF, como por ejemplo tokens de formulario.		**X**	
Verificar si se implementan mecanismos avanzados de verificación de operaciones y acciones críticas, como por ejemplo CAPTCHA o tarjetas de coordenadas.		**X**	
Verificar si se dispone de un sistema de gestión de versiones para la aplicación Web (así como un programa más general de gestión de cambios). Obtener detalles de este sistema y de cómo se emplea en el control de vulnerabilidades de seguridad y en la generación de actualizaciones y nuevas versiones.	**X**	**X**	
Comprobar si existe un criterio para la utilización de métodos HTTP GET y POST en los diferentes componentes de la aplicación Web.	**X**	**X**	
Comprobar si se emplean las cabeceras HTTP por parte de la aplicación Web como mecanismo de envío de información confidencial o de verificación del estado del cliente.	**X**	**X**	
Verificar si se hace uso de peticiones y capacidades asíncronas, AJAX, en la aplicación Web, así como los mecanismos de seguridad empleados para proteger este tipo de comunicaciones.	**X**	**X**	
Identificar los mecanismos y tecnologías que se emplean para la autentificación de usuarios: métodos de autentificación HTTP básico o digest, NTLM, autentificación mediante certificados SSL (cliente y/o servidor), autentificación mediante formularios y base de datos, o autentificación de múltiples factores.	**X**	**X**	
Identificar si se hace uso de las capacidades de autentificación del entorno de desarrollo Web empleado. Obtener detalles sobre los mecanismos de autentificación empleados.	**X**	**X**	

	CATEGORÍA DEL SISTEMA		
	MEDIA	**ALTA**	**COMPROBACIÓN**
Identificar qué mecanismos y tecnologías se emplean para el mantenimiento de sesiones de usuario: URLs, cookies, cabeceras HTTP propietarias, etc.	**X**	**X**	
Verificar si se hace uso de las capacidades de mantenimiento de sesiones del entorno de desarrollo Web empleado. Obtener detalles sobre los mecanismos de gestión de sesiones empleados.	**X**	**X**	
Verificar cómo se lleva a cabo la gestión de errores de las tres capas: servidor Web, de aplicación y de base de datos, así como si existe una librería centralizada para la gestión y control de los errores.		**X**	
Identificar cómo se realiza la gestión del código fuente de la aplicación en el entorno de producción, así como de los recursos existentes por defecto en el software empleado, con el objetivo de minimizar la información disponible.		**X**	
Identificar si se realiza algún tipo de comprobación para asegurarse de que no existe ningún tipo de información confidencial, tal como campos ocultos en documentos HTML o claves de la aplicación, en todos los documentos y datos enviados a los clientes Web.	**X**	**X**	
Verificar qué mecanismos se han implantado a nivel de la aplicación Web para la generación, almacenamiento y gestión de logs, así como si se dispone de una librería única y centralizada para la gestión de logs en la totalidad de la aplicación.	**X**	**X**	
Identificar qué entorno de generación y gestión de logs se ha utilizado para implementar el mecanismo de logging de la aplicación Web	**X**	**X**	
Identificar qué flexibilidad existe para determinar de qué eventos generar logs, así como el nivel de detalle de los logs. Obtener un listado detallado con todos los campos estándar de cualquier entrada de log estándar.		**X**	
AUDITORÍAS DE SEGURIDAD DE LA APLICACIÓN WEB			
Verificar si se realiza, o se ha planificado la realización de, una auditoría de seguridad sobre la aplicación Web una vez completado su desarrollo y antes de su puesta en producción. Identificar el tipo de auditoría: caja blanca, negra o ambas.	**X**	**X**	
Verificar si se realizan auditorías de seguridad sobre la aplicación Web durante las fases de desarrollo. Identificar el tipo (caja blanca, negra o ambas) y con qué frecuencia se realizan.	**X**	**X**	
Para las auditorías de caja negra, verificar si se emplean herramientas automáticas, pruebas manuales o ambas.	**X**	**X**	

	CATEGORÍA DEL SISTEMA		
	MEDIA	**ALTA**	**COMPROBACIÓN**
Identificar la metodología que se sigue en las auditorías de caja negra.		**X**	
Contrastar la metodología y elementos de la auditoría de caja negra con el listado de recomendaciones proporcionado en el apartado “ESPECIFICACIÓN DE REQUISITOS DE AUDITORÍAS DE SEGURIDAD DE ENTORNOS WEB” de la Guía de Seguridad de las TIC CCN-STIC-812		**X**	
Para las auditorías de caja blanca, verificar si se emplean herramientas automáticas, pruebas manuales o ambas.		**X**	
Identificar la metodología que se sigue en las auditorías de caja blanca.		**X**	
Identificar qué herramientas de tipo WASS se emplean y el por qué, así como cuáles han sido los criterios empleados para su elección.		**X**	
Identificar qué herramientas de análisis de código se emplean y el por qué, así como cuáles han sido los criterios empleados para su elección.		**X**	
Identificar qué pautas, cláusulas y requisitos se aplican para la evaluación y contratación de una aplicación Web a un tercero.	**X**	**X**	
FORMACIÓN Y CONCIENCIACIÓN			
Se deberán realizar cursos de formación de seguridad centrados en proporcionar un conocimiento adecuado a administradores y desarrolladores respecto a las vulnerabilidades y amenazas de seguridad en entornos Web, los diferentes tipos de ataques existentes y los mecanismos de defensa asociados.	**X**	**X**	
Se deberán realizar tareas de concienciación en materia de seguridad, relativas al desarrollo seguro de aplicaciones y servicios Web.	**X**	**X**	